Text Classification Homework

Bag-of-Words · Multinomial Naive Bayes · Logistic Regression

NAME & SURNAME
Enis Ogdum

STUDENT NUMBER
23011056

Part A – Text Classification with Bag-of-Words and Multinomial Naive Bayes

A1. Vocabulary and Bag-of-Words Representations

Vocabulary: V = (good, fun, great, nice, bad, boring, awful, slow)

ID	Training Text	Class	Bag-of-Words Representation
1	good fun	positive	[1, 1, 0, 0, 0, 0, 0, 0]
2	great nice	positive	[0, 0, 1, 1, 0, 0, 0, 0]
3	good great fun	positive	[1, 1, 1, 0, 0, 0, 0, 0]
4	bad boring	negative	[0, 0, 0, 0, 1, 1, 0, 0]
5	awful bad	negative	[0, 0, 0, 0, 1, 0, 1, 0]
6	slow boring bad	negative	[0, 0, 0, 0, 1, 1, 0, 1]

A2. Class Priors

Total documents N = 6 | Positive: $N_{pos} = 3$ | Negative: $N_{neg} = 3$

Class	Documents	Total	Prior
positive	3	6	0.5
negative	3	6	0.5

A3. Smoothed Conditional Probabilities

Total words in positive class = 7 | Total words in negative class = 7 | $|V|$ = 8 | Laplace $\alpha = 1$

Token	Count (pos)	P(w|positive)	Count (neg)	P(w|negative)
good	2	(2+1)/15 = 0.2000	0	(0+1)/15 = 0.0667
fun	2	(2+1)/15 = 0.2000	0	(0+1)/15 = 0.0667
great	2	(2+1)/15 = 0.2000	0	(0+1)/15 = 0.0667
nice	1	(1+1)/15 = 0.1333	0	(0+1)/15 = 0.0667
bad	0	(0+1)/15 = 0.0667	3	(3+1)/15 = 0.2667
boring	0	(0+1)/15 = 0.0667	2	(2+1)/15 = 0.2000
awful	0	(0+1)/15 = 0.0667	1	(1+1)/15 = 0.1333
slow	0	(0+1)/15 = 0.0667	1	(1+1)/15 = 0.1333

A4. Predictions for Test Documents

$x^{(1)}$ = "good great"
BoW: [1, 0, 1, 0, 0, 0, 0, 0]
score(pos) = 0.5 × 0.2000 × 0.2000 = **0.0200**
score(neg) = 0.5 × 0.0667 × 0.0667 ≈ **0.0022**
Predicted class: positive (0.0200 > 0.0022)

$x^{(2)}$ = "aweful boring"
BoW: [0, 0, 0, 0, 0, 1, 0, 0] *("aweful" is OOV and ignored)*
score(pos) = 0.5 × 0.0667 ≈ **0.0333**
score(neg) = 0.5 × 0.2000 = **0.1000**
Predicted class: negative (0.1000 > 0.0333)

$x^{(3)}$ = "good slow new"
BoW: [1, 0, 0, 0, 0, 0, 0, 1] *("new" is OOV and ignored)*

score(pos) = 0.5 × 0.2000 × 0.0667 ≈ **0.0067**
score(neg) = 0.5 × 0.0667 × 0.1333 ≈ **0.0044**
Predicted class: positive (0.0067 > 0.0044)

A5. Conceptual Explanation of Fit and Predict

During the **fit** step, the model computes and stores the prior probabilities for each class and the conditional probabilities for each word in the vocabulary given each class.

- **Why Bag-of-Words ignores word order:** It strictly counts token frequencies regardless of sequence, treating the document as a multi-set of words.
- **What Laplace smoothing changes and why it is needed:** It adds pseudocounts to avoid zero probabilities for words unseen in a specific class during training; otherwise, one zero would annihilate the entire product probability.
- **Why the model can compare scores without the full posterior denominator:** The denominator P(x) is constant across all classes for a given input, so comparing numerators P(c) × P(x|c) preserves relative ranking perfectly.
- **How the naive independence assumption appears in the formula:** It appears in the product ∏ P(w|c), positing that each word's probability is independent of all other words given the class.

Part B – Sentiment Prediction with Feature Extraction and Logistic Regression

B1. Feature Extraction

Lexicons: POS = {good, great, fun, nice} | NEG = {bad, awful, boring, slow}
Features: f_1 = # positive words, f_2 = # negative words, f_3 = 1 if "!" else 0.

ID	Training Text	f_1	f_2	f_3	Label y
1	good fun !	2	0	1	1
2	great nice	2	0	0	1
3	good great fun!	3	0	1	1
4	bad boring	0	2	0	0
5	awful bad	0	2	0	0
6	slow boring bad	0	3	0	0

B2. Initial Predictions (w = 0, b = 0)

All $z_i = 0$, so $\sigma(0) = 0.5$ for every document.

ID	f_1	f_2	f_3	z_i	$\hat{y}_i$	y_i	$\hat{y}_i - y_i$
1	2	0	1	0	0.5	1	−0.5
2	2	0	0	0	0.5	1	−0.5
3	3	0	1	0	0.5	1	−0.5
4	0	2	0	0	0.5	0	+0.5
5	0	2	0	0	0.5	0	+0.5
6	0	3	0	0	0.5	0	+0.5

B3. Gradients (m = 6)

Quantity	Computation	Result
$\partial J/\partial w_1$	$1/6 \times [(-0.5)(2) + (-0.5)(2) + (-0.5)(3) + 0 + 0 + 0] = -3.5/6$	≈ -0.5833
$\partial J/\partial w_2$	$1/6 \times [0 + 0 + 0 + (0.5)(2) + (0.5)(2) + (0.5)(3)] = 3.5/6$	≈ 0.5833
$\partial J/\partial w_3$	$1/6 \times [(-0.5)(1) + 0 + (-0.5)(1) + 0 + 0 + 0] = -1.0/6$	≈ -0.1667
$\partial J/\partial b$	$1/6 \times [-0.5 - 0.5 - 0.5 + 0.5 + 0.5 + 0.5]$	$= 0$

B4. Updated Parameters (learning rate η = 1)

Parameter	Update Rule	New Value
w_1	$0 - 1 \times (-3.5/6)$	≈ 0.5833
w_2	$0 - 1 \times (3.5/6)$	≈ -0.5833
w_3	$0 - 1 \times (-1.0/6)$	≈ 0.1667
b	$0 - 1 \times 0$	$= 0$

B5. Predictions for Test Texts

$t^{(1)}$ = "good nice !"

Features: $f_1=2$, $f_2=0$, $f_3=1$

$z = (3.5/6)(2) + (-3.5/6)(0) + (1.0/6)(1) + 0 = 8/6 \approx 1.3333$

$\sigma(z) = 1/(1+e^{-1.3333}) \approx 0.791$

Predicted label: 1 (positive) since $0.791 \geq 0.5$

$t^{(2)}$ = "bad slow"

Features: $f_1=0$, $f_2=2$, $f_3=0$

$z = (3.5/6)(0) + (-3.5/6)(2) + (1.0/6)(0) + 0 = -7/6 \approx -1.1667$

$\sigma(z) = 1/(1+e^{1.1667}) \approx 0.237$

Predicted label: 0 (negative) since $0.237 < 0.5$

$t^{(3)}$ = "good bad"

Features: $f_1=1$, $f_2=1$, $f_3=0$

$z = (3.5/6)(1) + (-3.5/6)(1) + 0 = 0$

$\sigma(z) = 0.5$

Predicted label: 1 (positive) since $0.5 \geq 0.5$

B6. Conceptual Explanation of Fit and Predict

In logistic regression, the **fit** step learns numeric weights and a bias via gradient descent, minimizing a loss function. This differs from Naive Bayes, which stores probability count tables.

- **Why sigmoid outputs a probability:** The sigmoid bounds any real-valued dot product output within (0, 1), forming a valid probability.
- **What the sign of a weight means:** A positive weight increases the probability of class 1; a negative weight shifts predictions toward class 0.

- **How the bias changes the decision threshold:** The bias is a learned offset when all inputs are zero, effectively shifting the separating hyperplane.
- **Why feature extraction is necessary:** Logistic regression requires fixed-size numeric vectors for dot products; it cannot process raw text strings natively.

Control Questions

C1. What does Bag-of-Words preserve, and what does it lose?
It preserves vocabulary token frequencies within a document. It loses word order, grammatical structure, and semantic context.

C2. What does Multinomial Naive Bayes store after fit?
The prior probabilities P(c) for each class and the full matrix of conditional token probabilities P(w|c).

C3. Why is Laplace smoothing important?
Without it, words unseen in a class during training receive a zero probability, which would set the entire product probability to zero for any test sentence containing that word.

C4. Why can log-scores replace raw probability products?
The logarithm is a strictly monotonic transformation, converting tiny multiplications into manageable sums without disturbing relative class ranking, and eliminating floating-point underflow.

C5. Why must the vocabulary be fixed after training?
The vocabulary defines the feature space dimensions. Allowing changes would misalign feature indices between training parameters and test inputs.

C6. What does logistic regression store after fit?
The optimized numeric weights w_j for each feature and the scalar bias term b.

C7. Why must text be transformed to numeric features before logistic regression?
Logistic regression operates through linear algebra (matrix-vector products), which requires identically sized continuous numeric inputs.

C8. What is the role of the sigmoid function?
It maps an unbounded linear combination z to the range (0, 1), producing a valid class membership probability.

C9. What do positive and negative weights mean?
A positive weight increases the score toward class 1; a negative weight decreases it toward class 0.

C10. Why is one gradient descent step enough for homework but not real training?
One step illustrates the mechanics of computing and applying gradients. Real-world loss landscapes require many iterations across hundreds of epochs to converge to a generalized minimum.

Final Comparison of the Two Models

Multinomial Naive Bayes is a generative model. In its **fit** stage it counts token occurrences to build probability tables. In its **predict** stage it multiplies priors by conditional word probabilities using Bayes' theorem.

Logistic Regression is a discriminative model. In its **fit** stage it iteratively adjusts feature weights by minimizing a loss function via gradient descent. In its **predict** stage it computes a weighted sum of numeric features, passes it through the sigmoid function, and outputs a class probability.

Despite their architectural differences, both models share the same prerequisite: raw text must be converted to fixed-size numeric representations before any classification can occur.